

Oficina: Como Replicar o Projeto e Montar um Hub de Simulações

Objetivo da oficina

Capacitar professores para:

- entender a arquitetura do repositório;
- criar ou adaptar simulações com a base já pronta;
- configurar o fluxo seguro de estudantes (planilha + Apps Script);
- adotar boas práticas de governança, privacidade e publicação.

Público e duração sugerida

- Público: professores da educação básica (perfil iniciante em programação).
- Duração total: 2h30.

Roteiro sugerido:

1. Visão geral do projeto (20 min)
2. Arquitetura do repositório (30 min)
3. Arquivos iniciais obrigatórios (30 min)
4. Planilha + script seguro de alunos (45 min)
5. Orientações gerais e próximos passos (25 min)

1) Arquitetura do repositório

Mapa mental rápido

```
simulacoes/
|-- index.html           # Vitrine principal do hub
|-- README.md           # Guia geral do projeto
|-- _shared/
|   |-- simulation-enhancer.js  # Núcleo comum: exercícios, envio, áudio,
UX                             # Configuração de acesso protegido de
|   |-- student-access-config.js
estudantes
|   |-- enem-question-bank.js  # Banco de questões (opcional por
simulação)
|-- docs/               # Governança, LGPD, checklists e método
|-- AvaliacaoBimestralEducacaoDigital/
|   |-- index.html       # Avaliação principal
|   |-- app.js           # Lógica da avaliação
|   |-- alunos.js        # Base local/fallback de estudantes
|   |-- STUDENT_DATABASE.js  # Variante de base estruturada
|   |-- google-apps-script.gs  # API de notas e nomes no Google Apps
Script
```

```
| `-- sync_alunos_js.py           # Sincroniza alunos.js com Apps Script  
|-- NomeDaSimulacao/  
| `-- NomeDaSimulacao.html       # Cada simulação em sua própria pasta  
`-- ... (demais simulações por tema)
```

Como explicar para colegas (fala sugerida)

"Nosso projeto tem uma lógica de plataforma: cada simulação vive em sua pasta, mas todas compartilham um mesmo motor comum em `_shared`. Isso evita retrabalho e garante padrão pedagógico."

Ideia-chave

- Escala com consistência: você adiciona novas simulações sem reescrever tudo.
- Padrão único de coleta e experiência do estudante.
- Governança centralizada em docs.

2) Arquivos iniciais necessários e características

Bloco A: Base institucional do hub

1. README.md

- Função: explicar propósito, metodologia e uso do projeto.
- Deve conter: objetivo didático, público, regras de privacidade e links de apoio.

2. index.html (raiz)

- Função: portal de entrada com cards das simulações.
- Característica: navegação por áreas/tópicos e fácil acesso para professores e alunos.

3. docs/

- Função: formalizar governança pedagógica e de dados.
- Essenciais para começar:
 - POLITICA-RETENCAO-DADOS.md
 - DICIONARIO-DE-DADOS.md
 - CHECKLIST-ANONIMIZACAO-E-PUBLICACAO.md

Bloco B: Motor técnico reutilizável

1. `_shared/simulation-enhancer.js`

- Função: camada comum das simulações.
- Entrega: persistência, exercícios, envio de resultados, fluxos de formulário, áudio e integrações.

2. `_shared/student-access-config.js`

- Função: parâmetros de acesso dos estudantes.
- Entrega: habilitação do modo protegido, hash, URL da API e timeout/cache.

3. _shared/enem-question-bank.js (opcional, recomendado)

- Função: banco de questões por simulationKey.
- Uso: incorporar prática ENEM de forma padronizada.

Bloco C: Pacote de avaliação/turma

1. AvaliacaoBimestralEducacaoDigital/app.js

- Função: aplicação da avaliação e integração com endpoint do Apps Script.

2. AvaliacaoBimestralEducacaoDigital/google-apps-script.gs

- Função: API de gravação de notas e consulta de alunos/abas.

3. AvaliacaoBimestralEducacaoDigital/alunos.js

- Função: fallback local e compatibilidade.
- Recomendação: manter vazio de nomes em produção pública quando API protegida estiver validada.

4. AvaliacaoBimestralEducacaoDigital/sync_alunos_js.py

- Função: sincronizar automaticamente alunos.js com o endpoint.
-

3) Planilha de alunos + script seguro (passo a passo)

3.1 Estrutura da planilha

Crie uma planilha Google com três abas:

- bySheet
- bySerieTurma
- byTrilha

Formato de cada aba:

- Coluna A: chave de busca.
- Coluna B em diante: nomes dos alunos (um por coluna).

Exemplos de chave:

- bySheet: 2o ano A
- bySerieTurma: 2o ano|A
- byTrilha: PCA - Educação Digital 3o ano E

3.2 Publicar o Apps Script

1. Abra a planilha e vá em Extensões > Apps Script.
2. Cole o conteúdo de google-apps-script.gs.
3. Ajuste no script:
 - SPREADSHEET_ID
 - ACCESS_TOKEN_HASH

4. Faça o deploy:
 - Implantar > Nova implantação > Aplicativo da Web
 - Executar como: você
 - Quem tem acesso: qualquer pessoa com o link
5. Copie a URL final /exec.

3.3 Conectar ao frontend

- Em app.js, atualizar APPS_SCRIPT_URL.
- Em _shared/student-access-config.js, definir:
 - rosterApiUrl
 - apiTimeoutMs
 - rosterCacheTtlMs

3.4 Fluxo seguro recomendado

1. Fase de transição:
 - manter fallback local;
 - validar API protegida em uso real.
2. Fase estável:
 - retirar nomes públicos de alunos.js;
 - manter apenas configurações mínimas de acesso;
 - deixar a lista nominal somente na planilha privada.

3.5 Boas práticas de segurança para a oficina

- Não compartilhar hash/token em slides públicos.
- Separar ambiente de teste e ambiente oficial.
- Definir rotação de senha por bimestre/semestre.
- Evitar expor dados individuais em relatórios.
- Trabalhar preferencialmente com indicadores agregados por turma/série.

3.6 Automação de atualização de lista

No diretório AvaliacaoBimestralEducacaoDigital, executar:

```
py -3 .\sync_alunos_js.py
```

Isso lê a URL no app.js, busca studentDatabase no endpoint e atualiza alunos.js.

4) Orientações gerais para replicação entre professores

Padrão pedagógico mínimo por simulação

Cada nova simulação deve ter:

- objetivo de aprendizagem explícito;
- experimento interativo principal;
- seção de exercícios com feedback;
- fechamento reflexivo (conclusão do estudante);
- envio de resultado para base de acompanhamento.

Padrão técnico mínimo

- Pasta própria para a simulação.
- Inclusão do enhancer no final da página.
- Definição de SIMULATION_ENHANCER_CONFIG com:
 - simulationName
 - storageKey

Exemplo:

```
<script>
window.SIMULATION_ENHANCER_CONFIG = {
  simulationName: "Nome da Simulação",
  storageKey: "nome-da-simulacao"
};
</script>
<script src="../../shared/simulation-enhancer.js"></script>
```

Governança e ética

- Usar checklist de anonimização antes de apresentar dados externos.
- Evitar qualquer exposição de desempenho individual.
- Priorizar linguagem: "uma escola pública de Brasília" em materiais acadêmicos externos.

Modelo de trabalho colaborativo entre professores

1. Professor autor: define objetivo e roteiro didático.
2. Professor validador: revisa linguagem, rigor conceitual e adequação ao ano/série.
3. Professor analista: acompanha indicadores agregados e propõe ajustes.
4. Ciclo mensal: testar, registrar, melhorar e republicar.

5) Checklist de implantação do hub na escola

1. Repositório publicado com index e README atualizados.
2. Pasta _shared consolidada e versão estável do enhancer.
3. Planilha Google estruturada (bySheet, bySerieTurma, byTrilha).
4. Apps Script implantado e testado.
5. student-access-config conectado ao endpoint correto.
6. Teste completo em modo estudante + modo visitante.
7. Checklist de anonimização aplicado antes de qualquer divulgação.
8. Rotina quinzenal de revisão pedagógica e técnica.

6) Roteiro de fala para apresentação (resumo de 5 minutos)

1. "Nós organizamos as simulações como plataforma, não como arquivos isolados."
 2. "A pasta _shared garante padrão único de experiência e coleta."
 3. "Cada simulação foca no conteúdo, enquanto o motor comum cuida da parte técnica repetitiva."
 4. "A lista de estudantes fica protegida via planilha + Apps Script, reduzindo exposição pública."
 5. "Trabalhamos com dados agregados por turma para apoiar decisões pedagógicas, com foco em ética e LGPD."
 6. "Nosso próximo passo é criar um hub colaborativo entre professores para coautoria e melhoria contínua."
-

7) Próximo passo prático da equipe

Reunião de 60 minutos com os colegas para:

1. escolher 2 simulações-piloto;
2. definir responsáveis por conteúdo e revisão;
3. configurar planilha e Apps Script em ambiente de teste;
4. aplicar com uma turma e comparar indicadores de adesão/conclusão.

Material já pronto para iniciar o piloto

- Esqueleto técnico: [templates/simulacao-esqueleto](#)
- Guia rápido de uso: [docs/GUIA-USO-ESQUELETO-SIMULACAO.md](#)
- Guia para cada professor criar o próprio sistema de notas: [docs/GUIA-CRIAR-SISTEMA-NOTAS-PROPRIO.md](#)
- Versão para impressão (folha de rosto + checklist de 1 página): [docs/SISTEMA-NOTAS-1PAGINA.md](#)

Arquivos principais do esqueleto:

- [templates/simulacao-esqueleto/SimulacaoModelo.html](#)
 - [templates/simulacao-esqueleto/exercicios-modelo.js](#)
 - [templates/simulacao-esqueleto/README.md](#)
-

8) Duas possibilidades de evolução do projeto

Você trouxe duas ideias muito boas. As duas são possíveis e, na prática, podem virar um plano em fases.

Opção 1: Esqueleto de simulação (rascunho sem conteúdo disciplinar)

O que é:

- Um pacote-base para qualquer professor começar rápido, sem precisar programar do zero.

O que incluir no esqueleto:

- Estrutura da pasta da simulação.
- HTML com layout padrão e seções prontas.

- Configuração mínima do enhancer.
- Bloco de exercícios com placeholders.
- Campos de conclusão e envio já conectados ao fluxo comum.
- README curto com instruções de preenchimento.

Vantagens:

- Implementação rápida.
- Formação docente mais simples.
- Padronização forte do hub.

Limites:

- Ainda depende de edição manual de conteúdo.
- Escala colaborativa cresce mais devagar.

Opção 2: Ambiente de produção com IA (pedido guiado de simulação)

O que é:

- Um site interno para o professor colaborador preencher um formulário e gerar uma primeira versão de simulação.

Fluxo proposto para o professor colaborador:

1. Escolher assunto.
2. Definir objetivos de aprendizagem.
3. Escolher tecnologia (Canvas, Three.js, PhET integrado, etc.).
4. Definir tipo e dinâmica dos exercícios.
5. Receber rascunho automático da simulação.
6. Revisar e publicar com curadoria pedagógica.

Arquitetura sugerida para esse ambiente:

- Frontend: formulário de pedido + painel de revisão.
- Backend: orquestrador que gera arquivos a partir de templates.
- IA: agente gerador de conteúdo técnico com regras pedagógicas.
- Repositório: grava em branch separada e abre PR para revisão humana.

Vantagens:

- Alta escala de produção.
- Entrada facilitada para professores externos.
- Acúmulo de inteligência coletiva no hub.

Riscos e cuidados:

- Exige governança de qualidade pedagógica.
- Precisa de revisão humana obrigatória antes de publicar.
- Deve evitar envio de dados sensíveis para API externa.

Recomendação prática (estratégia em fases)

Fase 1 (imediata):

- Implementar a Opção 1 (esqueleto) e validar com 3 professores.

Fase 2 (30 a 60 dias):

- Criar piloto da Opção 2 com geração assistida por IA apenas para rascunho técnico.

Fase 3 (após validação):

- Evoluir para plataforma completa com fluxo de revisão, versionamento e métricas.

Essa sequência reduz risco, acelera adesão da equipe e prepara o caminho para o hub colaborativo com IA.

Critérios para decidir em reunião

1. Tempo disponível da equipe nas próximas 6 semanas.
2. Número de professores colaboradores esperados.
3. Capacidade de revisão pedagógica antes da publicação.
4. Orçamento para API e infraestrutura.
5. Necessidade de escala imediata versus qualidade com crescimento gradual.